

Pseudo-Polynomial Time Algorithms for the Maximum-Density Subtree Problem and Related Problems

(Extended Abstract)

Sun-Yuan Hsieh* Ting-Yu Chou

Department of Computer Science and Information Engineering,
National Cheng Kung University,
No. 1, University Road, Tainan 701, TAIWAN

Abstract

Given a tree $T = (V, E)$ of n vertices such that each node v is associated with a value-weight pair (val_v, w_v) , where value val_v is a real number and weight w_v is a non-negative integer, the density of T is defined as $\frac{\sum_{v \in V} val_v}{\sum_{v \in V} w_v}$. A subtree of T is a connected subgraph (V', E') of T , where $V' \subseteq V$ and $E' \subseteq E$. Given two integers w_{min} and w_{max} , the weight-constrained maximum-density subtree problem on T is to find a maximum-density subtree $T' = (V', E')$ satisfying $w_{min} \leq \sum_{v \in V'} w_v \leq w_{max}$. In this paper, we present three pseudo-polynomial time algorithms for solving three different weight-constrained maximum-density subtree problems.

1 Introduction

Given a sequence S of n number pairs (a_i, w_i) with $w_i > 0$ for $i = 1, \dots, n$, and two weight bounds w_{min} and w_{max} , the *maximum-density segment problem* is to find a consecutive subsequence $S(i, j)$ of S such that the density of $S(i, j)$, i.e., $\frac{a_i + a_{i+1} + \dots + a_j}{w_i + w_{i+1} + \dots + w_j}$ is maximum over all other subsequence of S satisfying $w_{min} \leq w_i + w_{i+1} + \dots + w_j \leq w_{max}$. There are some invariants related to the maximum-density segment problem. Given a tree with weight and length on each edge, Wu, Chao, and Tang [4] presented an efficient algorithms for locating a maximum-weight path in a tree whose length is at most a given upper bound. Quite recently, Lin, Kuo, and Chao [1] studied the problem of finding a length-constrained

maximum-density path in a rooted tree with each edge is associated with a weight.

In this paper, we assume that each node v of a tree $T = (V, E)$ is associated with a *value-weight pair* (val_v, w_v) , where *value* val_v is a real number and *weight* w_v is a non-negative integer. The *density* of T , denoted by $d(T)$, is defined as $\frac{\sum_{v \in V} val_v}{\sum_{v \in V} w_v}$. The *weight-sum* of T , denoted by $w(T)$, is $\sum_{v \in V} w_v$. We consider the following three problems:

1. The weight-constrained maximum-density path problem. Given a tree T of n nodes associated with value-weight pairs, and two bounded integers w_{min} and w_{max} , find a maximum-density path P in T such that $w_{min} \leq w(P) \leq w_{max}$.
2. The weight-constrained maximum-density subtree problem. Given a tree T of n nodes associated with value-weight pairs, and two bounded integers w_{min} and w_{max} , find a maximum-density subtree T' in T such that $w_{min} \leq w(T') \leq w_{max}$.
3. The weight-constrained maximum-density Steiner tree problem. Given a tree T of n nodes associated with value-weight pairs, two bounded integers w_{min} and w_{max} , and a set of *terminals* $S \subset V(T)$, find a maximum-density subtree T' such that $S \subseteq V(T')$ and $w_{min} \leq w(T') \leq w_{max}$.

In this paper, we present three pseudo-polynomial algorithms for the above three problems. We show that the weight-constrained maximum-density path problem on trees can be solved in $O(w_{max}n)$ time. Based on this result, we also show that both the weight-constrained maximum-density subtree problem

*Corresponding author.
hsiehsy@mail.ncku.edu.tw

Email:

and the weight-constrained maximum-density Steiner tree problem on trees can be solved in $O(w_{\max}^2 n)$ time.

2 Preliminaries

A graph G is a pair (V, E) , where V and E are the vertex set and the edge set, respectively. We also use $V(G)$ and $E(G)$ to denote its vertex set and edge set, respectively. A path is a simple graph whose vertices can be ordered so that two vertices are adjacent if and only if they are consecutive in the list. A *subgraph* of $G = (V, E)$ is a graph (V', E') such that $V' \subseteq V$ and $E' \subseteq E$. An *induced subgraph* is an edge-preserving subgraph, that is, (V', E') is an induced subgraph of (V, E) iff $V' \subseteq V$ and $E' = \{(x, y) \in E \mid x, y \in V'\}$. For graph-theoretic terminologies and notations not mentioned here, readers should refer to [3].

A *tree* is a connected graph and has no cycles. A rooted tree is a tree with one node chosen as the root. For a node v in a rooted tree T with the root r , the *parent* of v , denoted by $\text{par}(v)$, is its neighbor on the unique path from v to r ; its *children*, denoted by $\text{child}_T(v)$, are its other neighbors; and its *descendants* are the nodes u such that the unique path from u to r contains v . The *leaves* are the nodes with no children. In contrast to the unrooted case, by the degree of a node v , denoted by $\text{deg}(v)$, we mean the number of children of v in the given rooted tree. For a node v in T , the *level* of v , denoted by $\text{level}(v)$, is the number of edges between v and the root.

For a tree T , a *subtree* of T is a connected subgraph (V', E') of T , where $V' \subseteq V$ and $E' \subseteq E$. For a rooted tree T , the *descendent-subtree rooted at v* is the subtree induced by all the descendants of v , rooted at v , which is denoted by T_v . Note that for a rooted tree T , every descendent-subtree rooted at v is a subtree rooted at v ; the converse is not always true. For a given node v in a rooted tree, the paths starting from v can be classified into two types: One is to stretch downward its children only, called *downward paths*, and the other is to include at least its parent, called *upward paths*. In the following sections, the input instance of the problem is restricted to be the rooted tree. If it is not the case, the tree can be easily transformed into a rooted one by selecting an arbitrary node as the root.

3 A weight-constrained maximum-density path

For a node v in a rooted tree T , we use $D_v^1[i]$ and $D_v^2[i]$ to denote the highest value and the second-best value of those downward paths starting from node v of weight i , respectively. We aim at computing $D_v^1[i]$ and $D_v^2[i]$ for each node v in T by bottom-up dynamic programming, where $i = w_v, w_v + 1, \dots, w_{\max}$. The function “max” always selects the maximum value of the downward paths. If there exist at least two maximum-value downward paths, the function “2nd-max” selects the maximum value; otherwise, it selects the second-best value. If v is a leaf, then $D_v^1[i]$ equals val_v if $i = w_v$, and equals $-\infty$ for otherwise. Moreover, we define $D_v^2[i]$ to be $-\infty$ for all $0 \leq i \leq w_{\max}$.¹

Suppose that the internal node v has m children $v_1, v_2, \dots, v_m$. Then, we have

$$D_v^1[i] = \max\{D_{v_j}^1[i - w_v] + \text{val}_v \mid 1 \leq j \leq m\}, \quad (1)$$

and

$$D_v^2[i] = \text{2nd-max}\{D_{v_j}^1[i - w_v] + \text{val}_v \mid 1 \leq j \leq m\}. \quad (2)$$

We can show the following lemma.

Lemma 1. *The downward entries D_v^1 and D_v^2 , for all nodes v in a rooted tree T can be computed in $O(w_{\max} n)$ time.*

The computation of upward entries is similar to that of downward entries. Let $U_v[i]$ denote the highest value of those upward paths starting from node v of weight i . For $i = w_v, w_v + 1, \dots, w_{\max}$, we compute $U_v[i]$ for each node v in tree T by a top-down dynamic programming. Initially, $U_r[i] = \text{val}_r$ if $i = w_r$, and $U_r[i] = -\infty$ if $i \neq w_r$.

Let $\text{par}(v)$ be the parent of node v . Then, we have

$$U_v[i] = \max \begin{cases} U_{\text{par}(v)}[i - w_v] + \text{val}_v, \\ D_{\text{par}(v)}^1[i - w_v] + \text{val}_v & \text{if } v \text{ isn't the} \\ & \text{contributor of } D_{\text{par}(v)}^1[i - w_v], \\ D_{\text{par}(v)}^2[i - w_v] + \text{val}_v & \text{if} \\ & D_{\text{par}(v)}^2[i - w_v] \text{ exists and } v \text{ is the} \\ & \text{contributor of } D_{\text{par}(v)}^1[i - w_v]. \end{cases} \quad (3)$$

¹For convenience, throughout this paper, we assume the value of an entry equals $-\infty$ if the value of such an entry is not assigned.

Lemma 2. *The upward entries U_v for all nodes v in a rooted tree T can be computed in $O(w_{\max}n)$ time.*

Now we present an algorithm shown in Figure 1 for computing the density of a maximum-density path under the weight-constraint.

Algorithm MDP($T, r, w_{\min}, w_{\max}$)

Input: A tree T rooted at r with n nodes, and two weight bounds $w_{\min}$ and $w_{\max}$.

Output: The density of a weight-constrained maximum-density path in T .

```

1: Compute level( $v$ )'s for all  $v \in V(T)$ 
2: for all  $v \in V(T)$  in decreasing order of level( $v$ ) do
3:   if  $v$  is a leaf then
4:      $D_v^1[w_v] \leftarrow val_v$ 
5:   else
6:     for  $i \leftarrow w_v$  to  $w_{\max}$  do
7:        $D_v^1[i] \leftarrow \max\{D_{v_j}^1[i - w_v] + val_v \mid v_j \text{ is a}$ 
           child of  $v\}$ 
8:        $D_v^2[i] \leftarrow 2\text{nd-max}\{D_{v_j}^1[i - w_v] + val_v \mid v_j \text{ is}$ 
           a child of  $v\}$ 
9: for all  $v \in V(T)$  in increasing order of level( $v$ ) do
10:  if  $v$  is the root then
11:     $U_v[w_v] \leftarrow val_v$ 
12:  else
13:    for  $i \leftarrow w_v$  to  $w_{\max}$  do
14:      if  $v$  isn't the contributor of  $D_{par(v)}^1[i - w_v]$ 
           then
15:         $U_v[i] \leftarrow \max\{U_{par(v)}[i - w_v] +$ 
            $val_v, D_{par(v)}^1[i - w_v] + val_v\}$ 
16:      else
17:         $U_v[i] \leftarrow \max\{U_{par(v)}[i - w_v] +$ 
            $val_v, D_{par(v)}^2[i - w_v] + val_v\}$ 
18: return  $\max_{v \in V(T)} \{ \max_{\substack{w_{\min} \leq i \leq w_{\max} \\ i \neq 0}} \{ \frac{D_v^1[i]}{i}, \frac{U_v[i]}{i} \} \}$ 

```

Figure 1: Computing the density of a weight-constrained maximum-density path on T .

Based on the information obtained after executing Algorithm MDP, we can also obtain an algorithm to construct a maximum-density path in $O(n)$ time. Therefore, the following result can be obtained.

Theorem 1. *The weight-constrained maximum-density path problem on trees can be solved in $O(w_{\max}n)$ time.*

4 A weight-constrained maximum-density subtree

In this section, we present an $O(w_{\max}^2n)$ -time algorithm using the dynamic-programming tech-

nique to solve the weight-constrained maximum-density subtree problem on trees.

Recall that $deg(v)$ is the number of children of a node v in a rooted tree T . For an internal node $v \in V(T)$ with children $v_1, v_2, \dots, v_{deg(v)}$, let $C_j^v[i]$ be the maximum total value of subtrees rooted at $v_1, v_2, \dots, v_j$, whose weights sum to i . Also let $B_v[i]$ be the maximum-value of a subtree of T_v rooted at v , with weight i . For a leaf v of T , we compute $B_v[i] = val_v$ if $i = w_v$; and $B_v[i] = -\infty$ for otherwise. For an internal node v with children $v_1, v_2, \dots, v_{deg(v)}$, we first compute $C_j^v[0] = 0$ and $C_j^v[i]$ for $i = 1, \dots, w_{\max} - w_v$ and $j = 1, 2, \dots, deg(v)$, as $C_j^v[i] =$

$$\begin{cases} \max_{0 \leq w \leq i} \{C_{j-1}^v[i - w] + B_{v_j}[w]\} & \text{if } 1 < j \leq deg(v), \\ B_{v_1}[i] & \text{if } j = 1. \end{cases} \quad (4)$$

After computing $C_{deg(v)}^v[w_{\max} - w_v]$, we obtain the maximum value of the subtrees rooted at all the children of v , whose weights from 0 to $w_{\max} - w_v$. We then compute $B_v[i]$ where $i = 0, 1, \dots, w_{\max}$ as follows:

$$B_v[i] = \begin{cases} C_{deg(v)}^v[i - w_v] + val_v & \text{if } w_v \leq i \leq w_{\max}, \\ 0 & \text{if } i = 0, \\ -\infty & \text{otherwise.} \end{cases} \quad (5)$$

Lemma 3. *All the entries $B_v[i]$, $C_j^v[i]$ can be computed in $O(w_{\max}^2n)$ time, where n is the number of nodes of the given tree T .*

An algorithm for computing the density of a weight-constrained maximum-density subtree in a rooted tree is presented in Figure 2.

Lemma 4. *The density of a weight-constrained maximum-density subtree in a rooted tree can be computed in $O(w_{\max}^2n)$ time.*

Theorem 2. *The weight-constrained maximum-density subtree problem on trees can be solved in $O(w_{\max}^2n)$ time.*

Algorithm $\text{MDT}(T, r, w_{\min}, w_{\max})$

Input: A tree T rooted at r with n nodes, and two weight bounds $w_{\min}$ and $w_{\max}$.

Output: The density of a weight-constrained maximum-density subtree in T .

```

1: compute level( $v$ )'s for all  $v \in V(T)$ 
2: for all  $v \in V(T)$  in decreasing order of level( $v$ ) do
3:   if  $v$  is a leaf node then
4:      $B_v[w_v] = val_v$ 
5:   else
6:     for  $j \leftarrow 1$  to  $\deg(v)$  do
7:       for  $i \leftarrow 0$  to  $w_{\max} - w_v$  do
8:         if  $j = 1$  then
9:            $C_1^v[i] \leftarrow B_{v_1}[i]$ 
10:        else
11:           $C_j^v[i] \leftarrow \max_{0 \leq w \leq i} \{C_{j-1}^v[i - w] + B_{v_j}[w]\}$ 
12:        for  $i \leftarrow w_v$  to  $w_{\max}$  do
13:           $B_v[i] \leftarrow C_{\deg(v)}^v[i - w_v] + val_v$ 
14: return  $\max_{v \in V(T)} \{ \max_{\substack{w_{\min} \leq i \leq w_{\max} \\ i \neq 0}} \{ \frac{B_v[i]}{i} \} \}$ 

```

Figure 2: An algorithm for computing the density of a weight-constrained maximum-density subtree in a rooted tree T .

5 A weight-constrained maximum-density Steiner tree

In this section, we consider the weight-constrained maximum-density Steiner tree problem. Given a set of terminals $S \subset V$ of a rooted tree $T = (V, E)$, a set $\mathcal{C}_S$ is a *connecting set* of S if $S \subseteq \mathcal{C}_S$ and the subgraph (i.e., subtree) of T induced by $\mathcal{C}_S$ is connected. The set $\mathcal{C}_S$ is further said to be *minimal* if $\mathcal{C}_S - v$ is not a connecting set of S for any node $v \in \mathcal{C}_S$. Note that if the subtree of T induced by S is connected, then $\mathcal{C}_S = S$. For a connecting set $\mathcal{C}_S$ of a rooted tree $T = (V, E)$, *contracting $\mathcal{C}_S$ to a supernode s* in T is to obtain another tree $T' = (V - \mathcal{C}_S + s, E')$, where E' is obtained by deleting $\{(x, y) \mid x \in \mathcal{C}_S \text{ or } y \in \mathcal{C}_S\}$ from E and adding one new edge (x, s) for each edge (x, y) , where $x \notin \mathcal{C}_S$ and $y \in \mathcal{C}_S$. Assume that $Q = \{(x, y) \mid x \notin \mathcal{C}_S \text{ and } y \in \mathcal{C}_S\}$. There are totally $|Q|$ created edges.

The key concept of our algorithm is first to root the given tree on an arbitrary node v in terminal set S . We then find a minimal connecting set $\mathcal{C}_S$, and contract it to a super-node s . Note that s is the root of the resulting tree T' . The value-weight pair associated with s is $(val_s, w_s) = (\sum_{v \in \mathcal{C}_S} val_v, \sum_{v \in \mathcal{C}_S} w_v)$. By executing Algorithm MDT on T' , we can find a weight-constrained maximum-density subtree rooted at s ,

denoted by T^s . A weight-constrained maximum-density Steiner tree is thus the subtree of T induced by $V(T^s) - s + \mathcal{C}_S$. Therefore, we have the following result.

Theorem 3. *The weight-constrained maximum-density Steiner tree problem on trees can be solved in $O(w_{\max}^2 n)$ time.*

References

- [1] R. R. Lin, W. H. Kuo, and K. M. Chao, "Finding a length-constrained maximum-density path in a tree," *Journal of Combinatorial Optimization*, 9(2):147–156, 2005.
- [2] M. Sipser, *Introduction to the Theory of Computation*, PWS Publishing Company, Boston, MA 02116, 1997.
- [3] D. B. West, *Introduction to Graph Theory*, 2nd ed, Prentice Hall, Upper Saddle River, NJ, 2001.
- [4] B. Y. Wu, K. M. Chao, and C. Y. Tang, "An efficient algorithm for the length-constrained heaviest path problem on a tree," *Information Processing Letters*, 69:63–67, 1999.